Data Engineering Interview Preparation

100 interview questions with answers + 15 practical coding questions and solutions

How to use this guide: Learn the short answer first, then practise explaining why and where you used the concept in a project. SQL examples use broadly supported syntax unless noted; some functions vary by database.

1. Data Engineering Fundamentals

1. What is Data Engineering?

Data Engineering is the discipline of designing, building and maintaining systems that collect, ingest, transform, store and serve data reliably for analytics and applications.

2. What is a data pipeline?

A data pipeline is a sequence of steps that moves data from sources to destinations, often including ingestion, validation, transformation, storage and monitoring.

3. What is ETL?

ETL means Extract, Transform, Load. Data is extracted from source systems, cleaned or transformed, and then loaded into a target such as a warehouse.

4. What is ELT?

ELT means Extract, Load, Transform. Raw data is loaded into the target platform first, then transformed using that platform's compute, for example SQL transformations in Snowflake.

5. ETL vs ELT?

ETL transforms before loading; ELT loads first and transforms inside the destination. ELT is common in cloud warehouses with scalable compute; ETL is useful when data must be transformed or masked before landing in the target.

6. What is batch processing?

Batch processing handles a bounded collection of records at scheduled intervals, such as processing the previous day's sales every night.

7. What is stream processing?

Stream processing handles events continuously or in small time windows as they arrive, such as detecting suspicious card transactions.

8. Batch vs streaming?

Batch is scheduled and processes accumulated data, often with higher latency. Streaming processes ongoing events with lower latency and requires attention to event time, late data, ordering and recovery.

9. What is data ingestion?

Data ingestion is collecting data from databases, files, APIs, applications or event streams and moving it into a storage or processing platform.

10. What is data transformation?

Transformation converts raw data into useful data by applying cleaning, validation, type conversion, standardization, enrichment, filtering and aggregation.

2. Data Warehousing and Architecture

11. What is a data warehouse?

A data warehouse is a system optimized for analytical queries and reporting. It stores integrated, historical and usually curated data. Examples include Snowflake and Amazon Redshift.

12. What is a data lake?

A data lake stores large amounts of data in its original or processed form, including structured, semi-structured and unstructured data. Examples include Amazon S3 and Azure Data Lake Storage Gen2.

13. Data lake vs data warehouse?

A lake is flexible and can retain raw files in many formats; a warehouse provides managed analytical tables and SQL-oriented querying. Many architectures use both.

14. What is a lakehouse?

A lakehouse combines low-cost, flexible data-lake storage with warehouse-like capabilities such as transactions, schema enforcement, governance and performant analytics. Delta Lake is one example.

15. What is Medallion Architecture?

It organizes data into Bronze (raw ingested data), Silver (cleaned, validated and standardized data) and Gold (business-ready aggregates or data products).

16. Schema-on-read vs schema-on-write?

Schema-on-read interprets structure when data is read; schema-on-write validates or applies structure before data is stored. Data lakes often support read-time flexibility, while warehouses commonly enforce schema on write.

17. What is a fact table?

A fact table records measurable business events, such as sales amount, quantity, order count or transaction value. Its grain should be clearly defined.

18. What is a dimension table?

A dimension table contains descriptive context for facts, such as customer, product, date, store or geography.

19. What is a star schema?

A star schema has a central fact table connected directly to denormalized dimension tables. It is designed to make analytical queries straightforward.

20. What is a snowflake schema?

A snowflake schema normalizes dimensions into related sub-dimension tables. It reduces repeated attributes but may require more joins.

3. Python for Data Engineering

21. Why is Python used in Data Engineering?

Python is readable and has a strong ecosystem for APIs, automation, cloud SDKs, data manipulation and distributed processing. Common tools include Pandas, PySpark, Boto3, Requests and SQLAlchemy.

22. What is Pandas?

Pandas is a Python library for working with tabular data using Series and DataFrames. It supports filtering, joining, grouping, missing-value handling and file I/O.

23. List vs tuple?

A list is mutable and written with square brackets; a tuple is immutable and written with parentheses. Tuples are useful for fixed collections and can be hashable when all their elements are hashable.

24. What is a generator?

A generator yields values lazily, one at a time, using yield. It can reduce memory use when processing large sequences.

25. Multithreading vs multiprocessing?

Threads share a process's memory and are useful for I/O-bound work. Processes have separate memory and can use multiple CPU cores, making them useful for CPU-bound Python work. Standard CPython's GIL limits parallel execution of Python bytecode across threads.

26. What is an iterator?

An iterator implements __iter__() and __next__() and returns values one at a time. Iterators support lazy processing.

27. How do you process a large CSV?

Read it in chunks, process each chunk, and write results incrementally rather than loading the entire file into memory. For distributed or very large workloads, use Spark or a managed ETL service.

28. What is Boto3?

Boto3 is the AWS SDK for Python. It lets Python applications call AWS services such as S3, Lambda, SQS and DynamoDB using clients or resource objects.

Example: Read a large CSV in chunks

Each chunk can be validated and written to a destination independently.

```
import pandas as pd

for chunk in pd.read_csv("large.csv", chunksize=10000):
    clean = chunk.drop_duplicates()
    clean = clean.dropna(subset=["id"])
    clean.to_csv("cleaned.csv", mode="a",
                 index=False, header=False)
```

Example: Upload a file to S3 with Boto3

The AWS identity used by the application must have permission to upload to the target bucket.

```
import boto3

s3 = boto3.client("s3")
s3.upload_file(
    "data.csv",
    "my-bucket",
    "raw/data.csv"
)
```

4. SQL for Data Engineering

29. WHERE vs HAVING?

WHERE filters rows before grouping. HAVING filters grouped results after GROUP BY.

30. What is a window function?

A window function calculates a value across related rows while keeping each input row in the result. Examples include ROW_NUMBER, RANK, LAG and running SUM.

31. ROW_NUMBER vs RANK vs DENSE_RANK?

ROW_NUMBER assigns a unique sequence. RANK gives tied rows the same rank and leaves gaps. DENSE_RANK gives tied rows the same rank without gaps.

32. What is a CTE?

A Common Table Expression is a named temporary result defined with WITH and used by one SQL statement. It can improve readability and support recursive queries in many databases.

33. What is an index?

An index is a data structure that can speed up data retrieval. It uses storage and can add overhead to inserts, updates and deletes. Index design should be based on query patterns and execution plans.

34. DELETE vs TRUNCATE vs DROP?

DELETE removes selected rows and supports WHERE. TRUNCATE removes all rows while retaining the table definition. DROP removes the table object. Transaction and identity behaviour varies by database.

35. What is UPSERT?

UPSERT inserts a row if its key does not exist and updates it if it does. It can be implemented with MERGE or database-specific conflict-handling syntax.

36. What is incremental loading?

Incremental loading processes only new or changed records since the previous successful run. Common approaches use timestamps, increasing IDs, log positions or CDC.

37. What is CDC?

Change Data Capture identifies inserts, updates and deletes in a source system and makes those changes available downstream, often using database logs or change tables.

Example: Department-wise employee count

GROUP BY creates one result per department.

```
SELECT department_id, COUNT(*) AS employee_count
FROM employees
GROUP BY department_id;
```

Example: Second-highest distinct salary

This portable pattern uses a subquery. It returns NULL if there is no second distinct salary.

```
SELECT MAX(salary) AS second_highest_salary
FROM employees
WHERE salary < (SELECT MAX(salary) FROM employees);
```

Example: Top 3 salaries per department

DENSE_RANK includes ties. Use ROW_NUMBER instead if exactly three rows per department are required.

```
WITH ranked AS (
    SELECT e.*,
           DENSE_RANK() OVER (
               PARTITION BY department_id
               ORDER BY salary DESC
           ) AS salary_rank
    FROM employees e
)
SELECT *
FROM ranked
WHERE salary_rank <= 3;
```

Example: Incremental load with a watermark

Store the last successfully processed timestamp and advance it only after the load commits. Exact syntax and concurrency controls depend on the database.

```
SELECT *
FROM source_orders
WHERE updated_at > :last_successful_watermark
  AND updated_at <= :current_watermark;
```

5. Apache Spark and PySpark

38. What is Apache Spark?

Apache Spark is a distributed processing engine for large-scale batch and streaming workloads, SQL, machine learning and graph processing.

39. What is PySpark?

PySpark is the Python API for Apache Spark. It lets Python programs use Spark's distributed execution engine.

40. What is an RDD?

An RDD (Resilient Distributed Dataset) is Spark's low-level distributed collection of objects. It supports transformations and actions and provides fine-grained control.

41. What is a DataFrame?

A Spark DataFrame is a distributed dataset organized into named columns with a schema. Spark can optimize DataFrame operations.

42. RDD vs DataFrame?

RDDs are lower-level collections of objects; DataFrames are structured and schema-based. DataFrames are usually preferred for ETL because Spark can optimize their execution with Catalyst.

43. What is lazy evaluation?

Spark records transformations in a logical plan and delays execution until an action is called. This lets Spark optimize the plan before running it.

44. Transformations vs actions?

Transformations create a new RDD or DataFrame, such as filter, select and join. Actions trigger execution or return/write results, such as count, show, collect and write.

45. What is partitioning?

A partition is a logical chunk of distributed data processed by a task. Partition count affects parallelism, scheduling overhead and data movement.

46. What is shuffling?

Shuffle redistributes data across partitions, often for groupBy, joins, distinct and repartition. It can be expensive due to network transfer, serialization and disk I/O.

47. repartition vs coalesce?

repartition can increase or decrease partitions and performs a shuffle to redistribute data. coalesce is commonly used to reduce partitions with less data movement, but may leave uneven partitions.

48. What is data skew?

Data skew occurs when some partitions contain much more data than others, creating slow straggler tasks. Techniques include salting keys, broadcast joins, better partitioning and skew-join optimizations.

49. What is a broadcast join?

Spark distributes a small table to executors so it can be joined locally with a larger table, often avoiding a shuffle of the large side. It is suitable only when the broadcast side safely fits in executor memory.

50. What is Delta Lake?

Delta Lake is a storage layer for data lakes that adds ACID transactions, schema enforcement and evolution, time travel and reliable update/delete/merge operations.

Example: Deduplicate and aggregate with PySpark

This example removes duplicate employee IDs and calculates average salary by department.

```
from pyspark.sql import functions as F

clean = df.dropDuplicates(["employee_id"])

result = (
    clean.groupBy("department_id")
         .agg(F.avg("salary").alias("avg_salary"))
)
result.show()
```

Example: Write Parquet output

Parquet is columnar and commonly used for analytical workloads. Choose partition columns based on common query filters and avoid creating too many tiny files.

```
(
    result.write
          .mode("overwrite")
          .parquet("s3://my-bucket/processed/department_salary/")
)
```

6. AWS Data Engineering

51. What is Amazon S3?

Amazon S3 is object storage used for raw files, processed datasets, logs, backups and data-lake storage. Objects are stored in buckets under keys.

52. What is AWS Lambda?

Lambda is an event-driven serverless compute service. It runs code without server management and scales automatically. Its maximum configured execution timeout is 15 minutes.

53. What is AWS Glue?

AWS Glue is a managed serverless data integration service for discovering, preparing, transforming and moving data. It supports Spark-based ETL jobs and other job types.

54. What is a Glue Crawler?

A crawler scans configured data sources, infers schema and creates or updates table metadata in the Glue Data Catalog.

55. What is Glue Data Catalog?

It is a centralized metadata repository containing table definitions, schemas, partitions and data locations. Services such as Athena and Glue can use it.

56. What is Amazon Redshift?

Redshift is a cloud data warehouse for analytical SQL workloads over large datasets. It supports columnar storage and massively parallel processing.

57. What is IAM?

AWS Identity and Access Management controls identities and permissions. Use least-privilege roles and policies rather than embedding long-lived credentials in code.

58. What is CloudWatch?

CloudWatch collects AWS metrics, logs and events and supports alarms and dashboards for monitoring workloads.

59. Lambda vs Glue?

Lambda is general-purpose serverless compute suited to short event-driven tasks; Glue is designed for data integration and ETL, including distributed Spark processing. Lambda has a 15-minute maximum timeout; Glue job limits depend on job type and configuration.

60. Explain S3 → Lambda → Glue → Redshift.

A file lands in S3; an event can invoke Lambda to validate metadata or start a Glue job; Glue transforms the data and writes curated output; the data is then loaded or made available to Redshift for analytics. Monitoring, retries and idempotency should be designed into the flow.

Example: Lambda handler for an S3 event

The execution role needs permission to read the source object and write logs. In production, validate the event and handle retries/idempotency.

```
def lambda_handler(event, context):
    for record in event["Records"]:
        bucket = record["s3"]["bucket"]["name"]
        key = record["s3"]["object"]["key"]
        print(f"New object: s3://{bucket}/{key}")

    return {"statusCode": 200}
```

7. Apache Airflow

61. What is Apache Airflow?

Airflow is a workflow orchestration platform used to define, schedule, execute and monitor workflows, including data pipelines.

62. What is a DAG?

A Directed Acyclic Graph defines tasks and dependencies. It has no directed cycles, so dependencies form a valid execution order.

63. Main Airflow components?

The scheduler parses DAGs and schedules tasks; the web UI provides visibility; an executor determines how tasks are run; workers execute tasks in distributed setups; the metadata database stores orchestration state.

64. What is an Operator?

An Operator defines a unit of work, such as running Python, executing SQL or invoking a shell command. Examples include PythonOperator and BashOperator.

65. What is XCom?

XCom lets tasks exchange small pieces of metadata or results. It is not intended for passing large datasets; store large data externally and pass a reference.

66. What is task dependency?

It defines execution order, for example extract >> transform >> load.

67. What happens when an Airflow task fails?

Airflow marks the task failed and can retry it according to its retry settings. Logs and task state help diagnose the issue; downstream tasks follow their trigger rules.

68. Airflow vs Cron?

Cron is a basic time-based scheduler. Airflow adds DAG dependencies, retries, task state, logs, monitoring and orchestration across complex workflows.

Example: Simple Airflow DAG

This illustrative DAG runs a Python task daily. Airflow versions differ in imports and recommended DAG authoring APIs.

```
from datetime import datetime
from airflow import DAG
from airflow.operators.python import PythonOperator

def extract():
    print("Extracting data")

with DAG(
    dag_id="daily_extract",
    start_date=datetime(2025, 1, 1),
    schedule="@daily",
    catchup=False
) as dag:
    extract_task = PythonOperator(
        task_id="extract",
        python_callable=extract
    )
```

8. Snowflake

69. What is Snowflake?

Snowflake is a cloud data platform for warehousing, data engineering, analytics and secure data sharing. It separates storage and compute.

70. What is a Virtual Warehouse?

A virtual warehouse is a compute cluster that executes queries, loads data and performs other compute operations. Warehouses can be resized and suspended/resumed.

71. What are Snowflake's architecture layers?

The main layers are database storage, query processing (virtual warehouses) and cloud services for metadata, authentication, optimization and coordination.

72. What is a Stage?

A stage is a location from which Snowflake loads files or to which it unloads files. Stages can be internal or external.

73. What is COPY INTO?

COPY INTO loads staged files into a table, or unloads table data to a stage, depending on the command form.

74. What is Snowpipe?

Snowpipe is Snowflake's continuous, event-driven data ingestion service for loading files as they arrive in a stage.

75. What is Time Travel?

Time Travel allows querying or restoring historical data within the retention period configured for the object and account edition.

76. What is Zero-Copy Cloning?

Cloning creates a database, schema or table copy using metadata references rather than immediately duplicating all underlying data. Subsequent changes are stored separately as needed.

Example: Load CSV from a stage

The file format and stage must be configured correctly, and the target table schema must match or be handled through explicit column mapping.

```
COPY INTO employees
FROM @my_stage
FILE_FORMAT = (TYPE = CSV SKIP_HEADER = 1);
```

9. Apache Kafka

77. What is Apache Kafka?

Kafka is a distributed event-streaming platform used to publish, store and consume streams of records.

78. What is a topic?

A topic is a named stream or category to which producers write records and consumers subscribe.

79. What is a partition?

A partition is an ordered, append-only log within a topic. Partitions allow parallelism and scale; ordering is guaranteed within a partition, not across the whole topic.

80. What is a consumer group?

A consumer group is a set of consumers that share work by reading topic partitions. Within a group, a partition is assigned to one consumer at a time.

81. What is an offset?

An offset is a record's position within a partition. Consumers track offsets to know their progress.

82. What is a producer?

A producer publishes records to Kafka topics, optionally choosing a key that influences partition assignment.

83. What is a consumer?

A consumer reads records from topic partitions and processes them.

84. What is a Dead Letter Queue?

A DLQ or dead-letter topic stores records that cannot be processed successfully after defined retries, allowing investigation and controlled reprocessing.

85. Kafka vs RabbitMQ?

Kafka is optimized for durable, partitioned event streams and replay. RabbitMQ is a message broker with flexible routing and queue-based delivery. The best choice depends on retention, replay, routing and throughput needs.

10. Databricks and Azure

86. What is Databricks?

Databricks is a data and AI platform built around Apache Spark and lakehouse technologies. It provides notebooks, jobs, governance and data engineering capabilities.

87. What is ADLS Gen2?

Azure Data Lake Storage Gen2 is Azure object storage with hierarchical namespace capabilities, designed for analytics and large-scale data lakes.

88. What is Unity Catalog?

Unity Catalog is Databricks' centralized governance layer for data and AI assets, providing permissions, discovery, lineage and auditing.

89. What is a Databricks Notebook?

A notebook is an interactive environment for developing and running code, often using Python, SQL, Scala or R, against a Databricks compute resource.

90. What is a Delta Table?

A Delta table is a table stored using Delta Lake format. It supports ACID transactions, schema controls, time travel and operations such as MERGE.

11. Real-world Scenario Questions

91. How do you handle duplicates?

Identify the business key and the rule for choosing the winning record (latest timestamp, highest version, etc.). Use SQL ROW_NUMBER or Spark dropDuplicates when appropriate, and make the load idempotent.

92. How do you handle NULL values?

Profile nulls, determine whether they mean unknown, not applicable or invalid, then apply business rules such as defaulting, quarantine or exclusion. Do not blindly replace every NULL with zero.

93. How do you handle bad or corrupted files?

Validate file type, schema, required columns and record quality. Quarantine invalid files or rows, record the reason and source, alert operators and provide a controlled reprocessing path.

94. How do you make a pipeline fault-tolerant?

Use retries with backoff, checkpoints where suitable, durable storage, monitoring, idempotent writes, dead-letter/quarantine paths and clear recovery procedures.

95. What is idempotency?

An operation is idempotent when repeating it produces the same final state as performing it once. For data loads, use stable keys, MERGE/upsert, deduplication and controlled batch identifiers.

96. How do you optimize a slow Spark job?

Inspect Spark UI and query plans; reduce unnecessary shuffles; use suitable broadcast joins; address skew; tune partitions; use columnar formats; filter early; avoid collect on large data; and reduce small-file problems.

97. How do you optimize a slow SQL query?

Inspect EXPLAIN or the query profile, check join strategy and filters, select only needed columns, use appropriate indexes or clustering where supported, reduce unnecessary scans and validate with representative data.

98. What is Parquet?

Parquet is a columnar file format with compression and encoding suited to analytical workloads. Readers can scan selected columns without reading every field.

99. CSV vs JSON vs Parquet?

CSV is simple tabular text; JSON supports nested semi-structured records; Parquet is a typed columnar binary format optimized for analytics and compression.

100. How do you explain an end-to-end project?

Cover the business problem, source systems, ingestion, raw storage, transformations, data quality, orchestration, target storage, monitoring, security, performance and business outcomes. Explain your own contribution and trade-offs.

12. Important Coding Questions for Data Engineers — with Answers

The following examples are practical starting points. Run them with sample data and adapt validation, error handling and database syntax to the actual environment.

1. Remove duplicates from a Python list

Use dict.fromkeys to preserve the original order.

```
values = [1, 2, 2, 3, 1, 4]
unique_values = list(dict.fromkeys(values))
print(unique_values)  # [1, 2, 3, 4]
```

2. Find the second-largest distinct number

Convert to a set to remove duplicates, sort, then select the second item. Handle lists with fewer than two distinct values.

```
numbers = [10, 5, 8, 10, 7]
unique = sorted(set(numbers), reverse=True)

second = unique[1] if len(unique) >= 2 else None
print(second)  # 8
```

3. Find the frequency of each word

Use Counter for concise frequency counting.

```
from collections import Counter

text = "data engineering data python"
counts = Counter(text.split())
print(counts)
# Counter({'data': 2, 'engineering': 1, 'python': 1})
```

4. Find common elements between two lists

Use set intersection when order and duplicate counts do not matter.

```
a = [1, 2, 3, 4]
b = [3, 4, 5, 6]

common = sorted(set(a) & set(b))
print(common)  # [3, 4]
```

5. Flatten a one-level nested list

Use a nested comprehension for a list nested exactly one level.

```
nested = [[1, 2], [3, 4], [5]]
flat = [item for row in nested for item in row]
print(flat)  # [1, 2, 3, 4, 5]
```

6. Read a large CSV in chunks

Chunking limits memory use. In production, write each processed chunk to a durable output and handle headers/append semantics carefully.

```
import pandas as pd

for chunk in pd.read_csv("large.csv", chunksize=50000):
    clean = chunk.drop_duplicates()
    clean.to_parquet("chunk_output.parquet")  # illustrative per-chunk output
```

7. Process JSON data using Python

Use json.load for a JSON file containing one JSON document. For newline-delimited JSON, parse each line separately or use a suitable reader.

```
import json

with open("input.json", encoding="utf-8") as f:
    records = json.load(f)

for record in records:
    print(record.get("id"), record.get("name"))
```

8. Handle missing values using Pandas

Inspect missingness first, then apply a business rule. This example fills a numeric column and removes rows missing a key.

```
import pandas as pd

df = pd.read_csv("employees.csv")
print(df.isna().sum())

df["salary"] = df["salary"].fillna(0)
df = df.dropna(subset=["employee_id"])
```

9. Find duplicate records using PySpark

Group by the business key and retain keys with more than one record.

```
from pyspark.sql import functions as F

duplicates = (
    df.groupBy("employee_id")
      .count()
      .filter(F.col("count") > 1)
)
duplicates.show()
```

10. Perform JOIN and GROUP BY using PySpark

Join on a common key, then aggregate by department.

```
from pyspark.sql import functions as F

result = (
    employees.join(departments, "department_id", "inner")
    .groupBy("department_name")
    .agg(F.count("*").alias("employee_count"))
)
result.show()
```

11. Find top 3 salaries by department using SQL

DENSE_RANK returns all ties within the top three distinct salary levels.

```
WITH ranked AS (
    SELECT e.*,
           DENSE_RANK() OVER (
               PARTITION BY department_id
               ORDER BY salary DESC
           ) AS rnk
    FROM employees e
)
SELECT *
FROM ranked
WHERE rnk <= 3;
```

12. Implement incremental loading using SQL

Use a persisted watermark and a fixed upper bound for each run. Advance the watermark only after the destination write succeeds.

```
SELECT *
FROM source_table
WHERE modified_at > :last_success_time
  AND modified_at <= :run_upper_bound;
```

13. Convert CSV to Parquet using PySpark

Read the CSV with an explicit schema in production, validate the data, then write Parquet.

```
df = (
    spark.read
         .option("header", True)
         .option("inferSchema", True)
         .csv("s3://bucket/raw/input.csv")
)

(df.write
   .mode("overwrite")
   .parquet("s3://bucket/processed/output/"))
```

14. Build a simple ETL pipeline using Python and SQL

This example separates extraction, transformation and loading. The load function is a placeholder for a parameterized database write.

```
import pandas as pd
from sqlalchemy import create_engine

source = pd.read_csv("orders.csv")          # Extract
clean = source.dropna(subset=["order_id"]) # Transform
clean = clean.drop_duplicates(["order_id"])

engine = create_engine(DB_URL)
clean.to_sql(
    "orders_staging",
    engine,
    if_exists="append",
    index=False
)                                           # Load
```

15. Read data from S3 and write transformed data back to S3

This local-Python example uses Boto3 and Pandas for a modest CSV. For large datasets, prefer Glue or Spark rather than downloading the whole object into Lambda memory.

```
import boto3
import pandas as pd
from io import BytesIO

s3 = boto3.client("s3")
obj = s3.get_object(Bucket="my-bucket", Key="raw/input.csv")
df = pd.read_csv(obj["Body"])

df = df.drop_duplicates()
df = df.dropna(subset=["id"])

buffer = BytesIO()
df.to_parquet(buffer, index=False)
buffer.seek(0)

s3.put_object(
    Bucket="my-bucket",
    Key="processed/output.parquet",
    Body=buffer.getvalue()
)
```

13. Project Explanation Framework

How should I explain my Data Engineering project in an interview?

Use this sequence: (1) business problem and expected outcome; (2) source systems and file formats; (3) ingestion method and schedule/triggers; (4) raw landing zone; (5) transformations and data-quality checks; (6) curated storage/warehouse; (7) orchestration and dependencies; (8) monitoring, retries and security; (9) performance and cost choices; (10) final dashboard or business value. Be precise about what you personally built.

What follow-up questions should I expect?

Expect questions about why you selected each service, how you handle schema changes, duplicates, late-arriving data, failed runs, backfills, incremental loads, partition strategy, data skew, access control, secrets, monitoring, cost optimization and how you validate output correctness.

14. Recommended Study Order

• SQL: joins, grouping, CTEs, window functions, incremental loads and MERGE.

• Python: collections, generators, file processing, APIs, exception handling and Boto3.

• Spark/PySpark: DataFrames, lazy evaluation, transformations/actions, partitions, shuffle, joins and skew.

• AWS: S3, IAM, Lambda, Glue, Data Catalog, Redshift and CloudWatch.

• Airflow: DAGs, operators, dependencies, retries, XCom and scheduling.

• Snowflake and Databricks: stages, COPY INTO, warehouses, Delta Lake and Unity Catalog.

• Projects: practise a clear 2–3 minute architecture explanation and defend design choices.

Interview reminder: Treat the code as templates, not copy-paste production solutions. Add schema validation, logging, secure credentials, retries, idempotency, tests and database-specific syntax as appropriate.